

SOLVO

Guida tecnica MVP · versione 0.2

Stato dell'implementazione al 9 settembre 2026

Gestionale di manutenzione e Ordini di Lavoro, con intake assistito, routing deterministico e Control Center in tempo reale.

Questa guida descrive il software attualmente realizzato. Il progetto iniziale v0.1 rimane conservato come riferimento storico; le sue previsioni di integrazione cloud non rappresentano funzionalità già disponibili. La sezione 12 separa esplicitamente il target AWS dal runtime corrente.

Principio operativo: l'AI propone una bozza; la persona la rivede e conferma; il backend applica le regole e persiste le modifiche.

Percorso di lettura

1. Scopo e perimetro consegnato
2. Stack e architettura corrente
3. Dominio ODL e stati
4. Assegnazioni e routing
5. Notifiche Telegram
6. Link tecnico e sicurezza
7. Intake testo e audio
8. Dashboard e identità visiva
9. Accesso smartphone con Quick Tunnel
10. Validazione e test
11. Avvio della demo locale
12. Target AWS e costi
13. Fonti e manutenzione del documento

1. Scopo e perimetro consegnato

SOLVO supporta la gestione di guasti e interventi di facility maintenance mediante ODL (Ordini di Lavoro). Collega raccolta della segnalazione, revisione dei dati, assegnazione tecnica e monitoraggio operativo.

Superficie	Funzioni attualmente disponibili
Intake della richiesta	Inserimento manuale, testo assistito e caricamento/registrazione audio nella pagina Nuovo ODL; revisione del form e conferma
Control Center operatore	Dashboard, elenco e dettaglio ODL, stati, assegnazioni, notifiche, solleciti e storico
Pagina tecnico mobile	Accesso tramite link firmato, dettagli intervento, accettazione e rifiuto con note facoltative
Tecnici	Consultazione della configurazione per categoria, ordine di escalation e ruolo di caposquadra

Le responsabilità di richiedente, operatore e tecnico descrivono i flussi del prodotto. Non esiste ancora un sistema di login o di autorizzazione per ruolo. L'intake è integrato nel frontend corrente, senza un portale richiedente autenticato separato.

Confini da dichiarare durante la demo

L'interfaccia operatore può applicare le transizioni verso EVASO e CHIUSO. La pagina tecnico consegnata offre accettazione e rifiuto: non comprende ancora la consuntivazione dell'intervento o il comando mobile di evasione. Il backend non verifica autore autenticato o dettagli del lavoro eseguito per la transizione a EVASO.

Il modello WorkOrder corrente non contiene origine TESTO/AUDIO, flag rischio persone o un campo diretto del tecnico assegnato. Il collegamento al tecnico passa attraverso Assignment. Non sono consegnati un trattamento operativo dedicato al rischio persone o una chiamata di emergenza integrata.

Il routing parte con un comando esplicito dell'operatore. Anche la notifica è esplicita. La mancata risposta viene dichiarata manualmente: non esiste uno scheduler che faccia avanzare la catena allo scadere di un timer.

Non sono previsti SLA, scadenze derivate da SLA o metriche di "tempo aperto". AWS non è stato distribuito.

2. Stack e architettura corrente

Livello	Tecnologia e impiego attuale
Frontend	React, TypeScript e Vite; form, viste operatore e pagina tecnico responsive

Livello	Tecnologia e impiego attuale
Backend	Python, FastAPI e Pydantic; REST, validazione, WebSocket e servizi applicativi
Persistenza	SQLAlchemy per modelli/query; Alembic per migrazioni
Database	PostgreSQL nel runtime locale
Realtime	FastAPI WebSocket e ConnectionManager in memoria
AI / trascrizione	Provider mock locali deterministici
Notifiche	Provider mock oppure Telegram Bot API tramite HTTPX

```

Browser operatore / intake / tecnico
      |
      React + Vite
    /api      /ws
      |
FastAPI: un solo processo applicativo
| dominio e servizi | provider
|                   |-- mock AI / trascrizione
|                   |-- mock / Telegram Bot API
+-- SQLAlchemy -- PostgreSQL
+-- ConnectionManager WebSocket in memoria

```

Il backend è un monolite modulare. `backend/app/domain` contiene le politiche di stato e routing; `services` orchestra casi d'uso e transazioni; `api` gestisce HTTP/WebSocket; `schemas` valida contratti; `models` e `db` gestiscono la persistenza. Le query sono nei servizi: non è stato introdotto un livello repository separato.

Le API correnti usano `/api`, non `/api/v1`. Vite inoltra `/api` e `/ws` allo stesso FastAPI. PostgreSQL e REST rimangono la fonte autorevole dei dati.

Realtime: una sola istanza

`/ws/work-orders` pubblica invalidazioni dopo il commit con `type`, `work_order_id` e timestamp UTC, senza contatti, note o token. Dashboard e dettaglio rileggono i dati; il dettaglio filtra gli eventi per ODL. La lista ODL e la pagina tecnico non sono sottoscritte al feed.

L'indicatore Live/Riconnessione/Offline accompagna il tentativo di riconnessione con attese da 3 a 15 secondi. Alla riconnessione avviene una nuova lettura; eventi ravvicinati sono raggruppati per 150 ms. Rimane disponibile il refresh manuale.

Il ConnectionManager è locale al processo: avviare un solo worker/istanza backend. Non esistono pub/sub condiviso, replay persistente o garanzia di consegna degli eventi. Un errore WebSocket non annulla una transazione già confermata.

3. Dominio ODL e stati

Campi WorkOrder effettivamente persistiti

Campo	Regola corrente
id	Identificativo intero generato dal database
code	Univoco e generato dal backend: SOLVO, data UTC e 16 caratteri esadecimali
created_at	Timestamp server di creazione
updated_at	Timestamp server di aggiornamento
user_first_name, user_last_name	Nome e cognome del richiedente, obbligatori
user_phone	Telefono del richiedente, obbligatorio
user_email	Email facoltativa, nullable
fault_address	Indirizzo del guasto, obbligatorio
category_id	Riferimento a una categoria esistente nel database
priority	Uno dei cinque valori esatti sotto riportati
description	Descrizione obbligatoria, revisionata prima della creazione
status	Stato ODL, inizialmente APERTO
reminders_count	Contatore inizialmente zero, incrementato atomicamente dai solleciti

Creazione e modifica generica accettano i dati del richiedente, indirizzo, categoria, priorità e descrizione. Codice, stato, contatore e timestamp non sono modificabili mediante il PATCH generico. Solo l'email può essere esplicitamente svuotata con null.

Priorità

Valore persistito/API	Colore badge
PROGRAMMABILE	#64748B
BASSA	#3B82F6
MEDIA	#F59E0B
ALTA	#F97316
URGENTE	#EF4444

Stati e transizioni consentite

Stato corrente	Stati successivi consentiti
APERTO	IN_CORSO, ANNULLATO
IN_CORSO	EVASO, ANNULLATO
EVASO	CHIUSO, IN_CORSO
CHIUSO	Nessuno
ANNULLATO	Nessuno

Una richiesta dello stesso stato restituisce successo senza duplicare eventi o aggiornare timestamp. Una transizione vietata restituisce 409; enum o dati non validi restituiscono 422. La riapertura da EVASO a IN_CORSO è prevista.

Categorie e dati di riferimento

Vetri, Climatizzazione, Riscaldamento, Ascensore, Rete, Elettrico, Edile, Idraulico, Serramenti, Antincendio, Sicurezza, Arredi, Altro.

Le categorie sono righe di database, non un enum nel codice. La validazione controlla l'esistenza della categoria; il modello non ha un flag "attiva". Il seed crea tre tecnici demo e un caposquadra per ciascuna delle 13 categorie, per 52 tecnici fittizi complessivi. Il routing non dipende da questo numero.

Storico e solleciti

Ogni sollecito ha timestamp e `created_by` riferito a un utente esistente. Inserimento del sollecito, incremento del contatore e storico avvengono nella stessa transazione. I solleciti sono ammessi in ogni stato; ciascun POST riuscito ne crea uno distinto.

Lo storico riceve eventi di creazione, cambiamento stato, assegnazione, accettazione, rifiuto, mancata risposta, escalation, notifica e sollecito. Evasione, chiusura e annullamento sono registrati come cambiamenti di stato. I servizi aggiungono eventi senza modificarli; il DELETE fisico dell'ODL elimina però anche lo storico per cascata. Non si tratta quindi di un archivio audit immutabile rispetto alla cancellazione dell'ODL.

4. Assegnazioni e routing

Assignment rappresenta un tentativo verso un tecnico e ha uno stato separato da WorkOrder: PENDING, ACCEPTED, REJECTED, NO_RESPONSE, ESCALATED. Mantiene il numero del tentativo, il tecnico, i timestamp e le eventuali `rejection_notes`.

Un'assegnazione REJECTED non rende l'ODL "rifiutato". Tale stato non esiste nel dominio WorkOrder. Il rifiuto riguarda quel candidato; il WorkOrder conserva il proprio stato mentre si apre il tentativo successivo.

Sequenza configurabile

Per la categoria dell'ODL si ordinano prima i tecnici normali per `escalation_order`, poi i capisquadra per lo stesso campo; l'ID risolve eventuali parità. Il seed dimostrativo dispone tre tecnici agli ordini 1-3 e un caposquadra all'ordine 4. Non esistono filtri di disponibilità o zona.

```
Avvio esplicito -> Tecnico 1: PENDING
Rifiuto / nessuna risposta -> Tecnico 2: PENDING
Rifiuto / nessuna risposta -> Tecnico 3: PENDING
Rifiuto / nessuna risposta -> Caposquadra: PENDING

Accettazione -> Assignment ACCEPTED + WorkOrder IN_CORSO
Escalation diretta -> caposquadra non ancora tentato
```

Comando	Effetto e limiti
Avvia assegnazione	Crea il tentativo 1 solo se non esiste una catena precedente
Accetta	Richiede il tentativo corrente PENDING; imposta ACCEPTED e ODL IN_CORSO
Rifiuta	Accetta solo note facoltative; registra REJECTED e crea il prossimo PENDING nella stessa transazione
Nessuna risposta	Comando manuale; registra NO_RESPONSE e apre il successore atomicamente
Escala al caposquadra	Seleziona un leader mai tentato; l'eventuale PENDING corrente diventa ESCALATED

Non si ritenta un tecnico già coinvolto nello stesso ODL. L'escalation diretta può iniziare anche senza precedenti tentativi e non richiede una particolare priorità. Dopo il leader non si torna ai tecnici normali saltati.

Se non esiste un successore, rifiuto o mancata risposta restituiscono 409 e lasciano intatto il precedente PENDING, comprese note e storico. Se manca un leader idoneo, anche l'escalation non modifica nulla. Una catena già avviata non può ripartire dal tentativo 1.

Le azioni su tentativi vecchi o già gestiti restituiscono 409 senza duplicazioni. CHIUSO e ANNULLATO bloccano le mutazioni di assegnazione. Negli altri stati sono ammesse; un'accettazione su EVASO riporta l'ODL a IN_CORSO. `sent_at` non dimostra l'invio di un messaggio; `responded_at` è valorizzato per accettazione/rifiuto.

API operative principali

```
POST /api/work-orders/{id}/assignments/start
GET  /api/work-orders/{id}/assignments
GET  /api/work-orders/{id}/assignments/current
POST /api/assignments/{id}/accept
POST /api/assignments/{id}/reject
POST /api/assignments/{id}/no-response
POST /api/work-orders/{id}/assignments/escalate-team-leader
```

La query `current` restituisce il `PENDING`, oppure l'ultimo tentativo se non ce n'è uno pendente. Rifiuto e mancata risposta restituiscono il tentativo appena concluso; la UI rilegge il corrente per mostrare il successore.

5. Notifiche Telegram

Telegram Bot API è il trasporto selezionato per le notifiche MVP/demo. Il provider viene scelto nel backend:

```
NOTIFICATION_PROVIDER=mock
```

Il mock è deterministico, non effettua chiamate di rete e restituisce uno stato `simulated` con il link tecnico. Per l'invio reale si seleziona invece:

```
NOTIFICATION_PROVIDER=telegram
```

`TELEGRAM_BOT_TOKEN` e `TELEGRAM_DEMO_CHAT_ID` devono essere configurati esclusivamente nell'ambiente backend. Non sono inclusi valori in questa guida. Il token del bot non viene mai esposto al frontend.

Invio esplicito e controllato

L'operatore preme **Invia Telegram** nel dettaglio ODL. L'endpoint `POST /api/assignments/{id}/notify` verifica che l'assegnazione sia quella corrente `PENDING` e che l'ODL non sia terminale. Creazione, rifiuto ed escalation non inviano notifiche automaticamente: anche il prossimo tecnico richiede un nuovo invio esplicito.

Il messaggio contiene `SOLVO`, codice ODL, priorità, categoria, indirizzo del guasto e link sicuro alla pagina tecnico. Non include i contatti del richiedente. Il backend chiama `sendMessage` via `HTTPS`, con timeout di 10 secondi e antepresa link disabilitata.

La risposta restituisce `provider`, identificativo messaggio, stato di `submission` e `action_url`. `submitted` significa accettazione da parte dell'API Telegram, non conferma di ricezione o lettura sul telefono. Lo storico registra simulazione/`submission` senza segreti, token o corpo del messaggio. Errori di provider/configurazione restituiscono 503 senza registrare un falso successo.

Destinazione demo

Tutti i tecnici selezionati dal routing usano un unico TELEGRAM_DEMO_CHAT_ID configurato. Il link continua a identificare la reale assegnazione. Il telefono del tecnico non viene convertito in una destinazione Telegram; non è stato modificato il modello Technician.

L'associazione di un'identità/canale di notifica verificato a ogni tecnico è evoluzione di produzione. Non sono implementati polling Telegram, webhook di ricevuta, callback inline o retry automatici. Ripetere notify può inviare nuovamente; submission esterna e commit del database non sono atomici e non si garantisce "exactly once".

6. Link tecnico e sicurezza

I link usano token firmati **HMAC-SHA256** con ASSIGNMENT_ACTION_SECRET, separato dal token Telegram. Il payload contiene versione, ID assegnazione e scadenza; è firmato, non cifrato, e non contiene dati personali. Token e digest non sono conservati nel database.

Il segreto richiede almeno 32 byte casuali, senza default utilizzabile. Configurazione assente o inadeguata blocca la generazione dei link con 503. TECHNICIAN_ACTION_TOKEN_TTL_MINUTES ha default 1440 minuti e intervallo 1-10080. Il controllo di scadenza e firma precede l'accesso ai dati; la rotazione del segreto invalida tutti i link precedenti.

```
/tecnico/assegnazione/{token}
GET /api/public/assignments/{token}
POST /api/public/assignments/{token}/accept
POST /api/public/assignments/{token}/reject
```

La pagina mostra dettagli dell'intervento, tecnico, badge e azioni **Accetta intervento** e **Rifiuta**. Il rifiuto presenta solo note facoltative, senza motivazione obbligatoria o enum di ragioni. Le azioni riusano gli stessi servizi e vincoli delle API operatore. Il rifiuto pubblico non espone dati o link del tecnico successivo.

Token alterati, scaduti o non validi e assegnazioni inesistenti producono 404; azioni duplicate o superate producono 409. Non esiste login nella pagina: chi possiede il link può consultare quell'assegnazione e agire finché è ammissibile.

Protezioni e limiti attuali

I segreti restano nell'ambiente backend; .env è ignorato da Git. I link firmati non devono finire in log o materiali condivisi; i percorsi con token richiedono redazione nei log di accesso del server/proxy. La SPA usa no-referrer e le risposte pubbliche no-store.

Le API operatore e il feed WebSocket restano senza autenticazione di produzione. La firma del link tecnico non protegge quelle API. L'ambiente demo deve contenere dati fittizi; chat Telegram condivisa e accesso temporaneo via tunnel sono scorciatoie di sviluppo. L'utente deve sempre confermare una bozza assistita prima della creazione ODL. Cognito, MFA, RBAC e accesso tecnico autenticato appartengono al target futuro.

7. Intake testo e audio

Testo assistito

POST /api/ai/work-order-draft riceve un oggetto con text, da 1 a 10000 caratteri, escludendo input vuoti o solo spazi. Restituisce esclusivamente una bozza modificabile con dati proposti, descrizione e avvisi; campi mancanti o ambigui rimangono da completare.

AI_PROVIDER=mock seleziona l'estrazione locale deterministica con parole chiave e campi espliciti. Non è un modello linguistico remoto. La categoria viene risolta sulle righe presenti nel database, senza ID fissi; priorità e output sono validati con Pydantic. Il provider non riceve accesso al database o comandi di workflow. L'orchestrazione legge le categorie ma non scrive ODL, assegnazioni o storico.

```
Testo -> mock di estrazione -> validazione e categorie
      -> bozza nel form -> revisione/modifica della persona
      -> Conferma e crea ODL -> POST /api/work-orders
```

La conferma finale usa il normale endpoint WorkOrder. Analisi e creazione sono richieste separate. In caso di errore il frontend conserva il testo e consente retry o completamento manuale; non crea automaticamente un ODL.

Amazon Bedrock rimane il target di produzione descritto nella roadmap. Il provider Bedrock non è implementato; selezionare un provider non supportato produce 503. Non si presenta il mock come inferenza Bedrock reale.

Audio: upload e MediaRecorder

POST /api/ai/work-order-draft-audio accetta multipart con campo audio e restituisce transcript e draft. La UI consente upload oppure registrazione mediante MediaRecorder, arresto e comando **Trascrivi e analizza**. La registrazione richiede permesso microfono e contesto sicuro HTTPS o localhost; l'upload rimane disponibile se la registrazione non è supportata.

TRANSCRIPTION_PROVIDER=mock restituisce il testo dimostrativo configurato tramite TRANSCRIPTION MOCK TEXT: **non riconosce il parlato del file ricevuto**. La bozza mostra un avviso di simulazione. Il transcript riutilizza esattamente la pipeline di draft del testo, con revisione e conferma obbligatorie.

Sono supportati WebM, WAV, MP3 e MP4 con i MIME previsti dall'API. Il limite predefinito è 10 MiB; il backend è configurabile tra 1 e 25 MiB, mentre il browser resta limitato a 10 MiB. File vuoto/input invalido produce 422, MIME non supportato 415, dimensione eccessiva 413 e indisponibilità provider 503.

L'audio è temporaneo: non viene creato un archivio permanente, un record audio o un oggetto S3. Gli upload vengono chiusi anche in caso di errore; la validazione MIME del mock non verifica il codec. Amazon Transcribe rimane un'integrazione futura, non una funzionalità già disponibile.

8. Dashboard e identità visiva

L'interfaccia **SOLVO Fresh** adotta sidebar con gradiente indaco, superfici bianche, bordi freddi e ombre leggere, tabelle compatte, badge arrotondati e azioni principali acqua. Poppins è distribuito localmente nel bundle. I layout si adattano a schermi piccoli e i controlli tecnico sono adatti al tocco.

Vista	Comportamento corrente
Dashboard	Cinque conteggi: totale, APERTO, IN_CORSO, URGENTE in tutti gli stati, EVASO + CHIUSO; ultimi otto ODL
Lista ODL	Filtri server per stato e priorità; accesso al dettaglio
Nuovo ODL	Un form condiviso per manuale, testo AI e audio; categorie configurate e conferma
Dettaglio ODL	Dati, transizioni consentite, solleciti, storico, tentativi e tecnico corrente/ultimo
Tecnici	Elenco in sola lettura con categoria, ordine escalation, ruolo e contatto
Pagina tecnico	Dettagli essenziali, accetta/rifiuta, note facoltative e feedback

Le viste mostrano nomi categoria invece di ID; i payload mantengono `category_id`. Stato e priorità hanno badge testuali oltre al colore. Impostazioni è un placeholder disabilitato.

Nel dettaglio sono presenti **Assegna tecnico**, **Invia Telegram**, **Nessuna risposta**, **Escala al caposquadra** e **Aggiungi sollecito**, secondo stato e configurazione. Accetta/rifiuta restano sulla pagina tecnico. Le azioni aggiornano ODL e attività; il realtime aggiorna Dashboard e dettaglio anche per modifiche provenienti dallo smartphone. Loading, errori inline, retry delle letture e indicatore di connessione rendono visibile lo stato operativo.

Riferimento SOLVO Fresh



La figura è il concept visivo originale, non una cattura del software consegnato. Nomi, valori e testi esemplificativi appartengono al mockup. Le voci illustrative di scadenza e i conteggi della figura non definiscono il dominio SOLVO. La dashboard corrente ha cinque card e il vocabolario ODL descritto in questa guida.

Token	Valore
Primary / Primary dark	#5B5FEF / #4548C9
Aqua / Cyan	#18BFAE / #38BDF8
Background / Surface	#F7F8FC / #FFFFFF
Border	#E7E9F2
Text primary / secondary	#182033 / #697386

I colori delle priorità sono riportati nella sezione 3. La gerarchia visuale usa spaziatura regolare, titoli leggibili, card arrotondate e focus da tastiera visibile. Il riferimento vincolante dei token è docs/DESIGN_SYSTEM.md.

9. Accesso smartphone con Cloudflare Quick Tunnel

Solo sviluppo/demo. Quick Tunnel rende raggiungibile temporaneamente il frontend locale da uno smartphone fisico, consentendo di aprire il link ricevuto in Telegram.

```
cloudflared tunnel --url http://127.0.0.1:5173
```

Il comando fornisce un URL HTTPS temporaneo della forma `https://...trycloudflare.com`. Configurare `TECHNICIAN_ACTION_BASE_URL` nel backend con l'URL appena generato, senza aggiungere il percorso tecnico, e riavviare FastAPI. Inviare una nuova notifica per ottenere il link con la nuova origine.

La configurazione Vite corrente mantiene host `127.0.0.1`, porta `5173` e `allowedHosts: ['.trycloudflare.com']`. Consente quindi i sottodomini del servizio per questa demo. Le istruzioni precedenti che richiedevano soltanto un hostname puntuale non descrivono questa piccola modifica successiva.

```
Smartphone -> HTTPS Quick Tunnel temporaneo -> Vite locale
| /api e /ws
+--> FastAPI unico -> PostgreSQL
```

Il PC deve mantenere attivi database, FastAPI, Vite e cloudflared. Il tunnel non garantisce uptime; il suo URL è temporaneo e può cambiare al riavvio. Quando cambia, aggiornare la base URL backend e inviare un nuovo link. Non è un servizio di hosting di produzione né un componente del target AWS.

L'esposizione comprende l'app demo e i proxy verso API operatore non autenticate. Usare dati fittizi e arrestare il tunnel al termine della prova. Nessun hostname temporaneo effettivo è incluso nel documento.

10. Validazione e test

Percorso end-to-end già validato su dispositivo reale

La validazione manuale sotto riportata è stata confermata per l'MVP prima di questa revisione documentale; non rappresenta una nuova esecuzione durante la generazione della guida.

Punto di osservazione	Esito confermato
PC operatore	Creazione WorkOrder, assegnazione del tecnico e invio Telegram
Smartphone fisico	Ricezione del messaggio, apertura del link SOLVO firmato tramite Quick Tunnel e consultazione dei dettagli
Smartphone fisico	Rifiuto dell'intervento e inserimento di note di rifiuto

Punto di osservazione	Esito confermato
Backend	Assignment passa a REJECTED, nota salvata, routing crea automaticamente il prossimo Assignment PENDING
UI operatore	Storico aggiornato, tecnico successivo visualizzato e cambiamenti riflessi nell'interfaccia

L'accettazione era già stata validata: azione del tecnico → Assignment ACCEPTED → WorkOrder IN_CORSO. Il rifiuto non cambia il WorkOrder in uno stato di rifiuto. Il nuovo tentativo non implica un invio automatico del messaggio al successore.

Ultimo stato verificato dei test automatici

Suite	Ultimo risultato verificato
Backend	215 tests passed
Frontend	58 tests passed

Questi sono i conteggi verificati forniti per la revisione. La successiva modifica Vite allowedHosts per Cloudflare è una piccola configurazione demo e non costituisce un nuovo conteggio di test. Le suite applicative non sono state rieseguite per questa modifica esclusivamente documentale.

I test backend usano SQLite isolato per le API: non dimostrano il comportamento concorrente dei lock PostgreSQL. I test dei provider non inviano messaggi reali. La prova manuale su smartphone integra queste verifiche, senza attestare deployment AWS, trascrizione reale, autenticazione di produzione o evasione dalla pagina tecnico.

Comandi di verifica disponibili

```
# Dalla radice del repository
backend/.venv/bin/python -m pytest backend/tests -q

# Da frontend/
npm run typecheck
npm test
npm run build
```

Non risultano configurati strumenti backend di formatting, lint o type checking. Questa revisione controlla il documento, i riferimenti obsoleti, l'assenza di segreti, l'integrità della v0.1 e `git diff --check`; non modifica codice applicativo.

11. Avvio della demo locale

Prerequisiti: PostgreSQL attivo e database creato; dipendenze backend già installate nell'ambiente virtuale; pacchetti frontend installati; Node.js compatibile con il progetto (22.12+); cloudflared già disponibile se si usa lo smartphone. Non occorrono credenziali AWS.

Configurazione backend

Creare il `.env` locale sulla base di `.env.example`, senza versionarlo. Le variabili sotto indicano cosa configurare; non riportano credenziali o destinazioni reali.

Variabile	Configurazione richiesta
DATABASE_URL	Connessione al PostgreSQL locale, con credenziali solo nell'ambiente
ASSIGNMENT_ACTION_SECRET	Segreto casuale locale di almeno 32 byte; nessun valore pubblicato
TECHNICIAN_ACTION_BASE_URL	Origine frontend localhost sul PC oppure HTTPS temporaneo generato dal tunnel
NOTIFICATION_PROVIDER	mock per simulazione oppure telegram per invio reale
TELEGRAM_BOT_TOKEN	Token bot, solo backend; richiesto in modalità Telegram
TELEGRAM_DEMO_CHAT_ID	Destinazione demo unica, solo backend; richiesta in modalità Telegram
AI_PROVIDER	mock
TRANSCRIPTION_PROVIDER	mock

Opzioni aggiuntive: `TECHNICIAN_ACTION_TOKEN_TTL_MINUTES` (default 1440), `TRANSCRIPTION MOCK_TEXT` (testo dimostrativo), `MAX_AUDIO_UPLOAD_MB` (default 10). Riavviare FastAPI dopo ogni modifica delle impostazioni.

Il bot deve essere stato creato e avviato nella chat demo sul telefono. Il recupero della destinazione si svolge lato backend; token e chat ID non vanno inseriti nel frontend, nei log condivisi o negli screenshot.

Sequenza di avvio

Da backend/, con ambiente virtuale attivato e PostgreSQL disponibile:

```
alembic upgrade head
python -m app.scripts.seed_demo
uvicorn app.main:app --reload
```

Il seed è esplicito, non viene eseguito all'avvio e non crea ODL. Crea categorie, tecnici fittizi e un utente demo per i solleciti. Le esecuzioni sequenziali riusano i dati esistenti; conflitti di configurazione causano rollback. Eseguirlo una volta alla volta.

Da frontend/, in un altro terminale:

```
npm run dev
```

Aprire `http://127.0.0.1:5173`. frontend/.env può configurare `VITE_API_BASE_URL` per il proxy (default `http://127.0.0.1:8000`). Per **Aggiungi sollecito**, impostare `VITE_DEMO_USER_ID` all'ID effettivo stampato dal seed, quindi riavviare Vite. È attribuzione demo pubblica, non un'identità autenticata; in assenza di configurazione valida il pulsante è disabilitato.

Per il telefono, in un terzo terminale:

```
cloudflared tunnel --url http://127.0.0.1:5173
```

Configurare la base URL come nella sezione 9, selezionare Telegram nel backend e riavviarlo. La modalità reload è solo sviluppo: mantenere un singolo worker applicativo e non avviare istanze backend multiple.

Traccia demo di 2-3 minuti

1. Aprire Nuovo ODL; mostrare il form manuale o la bozza da testo/audio mock e la revisione obbligatoria.
2. Confermare la creazione; aprire il dettaglio dell'ODL APERTO.
3. Premere Assegna tecnico, poi Invia Telegram.
4. Sul telefono aprire il link, mostrare i dettagli e rifiutare con note facoltative.
5. Sul PC mostrare storico, tentativo REJECTED e successore PENDING aggiornati.
6. Se si vuole mostrare anche l'accettazione, inviare esplicitamente la notifica del tentativo successivo e accettarlo: l'ODL passa a IN_CORSO.

API health su `/health` e documentazione interattiva FastAPI su `/docs`. Non usare nella presentazione valori reali di contatto, credenziali o link firmati riutilizzabili.

12. Target AWS e costi

TARGET CLOUD: non implementato e non distribuito. Questa sezione riprende l'evoluzione architetturale richiesta, distinta dal runtime locale. La pianificazione rimane in `docs/ROADMAP.md`.

Componente target	Destinazione prevista
Frontend React statico	S3 + CloudFront
Backend FastAPI e WebSocket	EC2, inizialmente una sola istanza
PostgreSQL	Amazon RDS
Audio	S3 per storage oggetti
Interpretazione AI	Amazon Bedrock dietro il provider esistente
Trascrizione	Amazon Transcribe
Permessi dei servizi	IAM con privilegi minimi
Segreti	AWS Secrets Manager
Log e osservabilità	CloudWatch
Controllo della spesa	AWS Budgets

TARGET FUTURO

```

Browser -> S3 / CloudFront -> EC2 FastAPI -> RDS PostgreSQL
      |-> S3 audio -> Transcribe
      |-> Bedrock -> bozza da confermare
      +-> provider di notifica

```

Per la scala futura: Application Load Balancer (ALB), Auto Scaling Group, distribuzione multi-AZ e Redis/ElastiCache pub-sub per condividere gli eventi WebSocket tra processi/istanze. Servizi event-driven, per esempio SQS/EventBridge, potranno disaccoppiare elaborazioni e notifiche. Nessuno di questi componenti è presente nel runtime corrente.

L'evoluzione di sicurezza comprende Cognito, MFA, RBAC e accesso tecnico autenticato, insieme a identità/canali di notifica per ciascun tecnico e controlli di produzione sui dati personali. Non viene introdotta alcuna risorsa AWS da questa revisione.

Posizionamento economico della demo

Telegram Bot API è stato scelto come trasporto notifiche a costo zero per l'uso MVP/demo. Cloudflare Quick Tunnel è stato scelto per la connettività temporanea gratuita della demo. Non sostituisce un hosting di produzione e richiede che il PC locale resti operativo.

AI e trascrizione mock non consumano servizi cloud. Un futuro ambiente AWS, anche solo dimostrativo, può generare costi per compute, database, storage, rete e servizi AI. Questa guida non ripropone preventivi mensili o crediti promozionali della v0.1 come se fossero attuali; il dimensionamento e la stima AWS andranno verificati al momento del deployment.

13. Fonti e manutenzione del documento

La v0.2 è stata redatta leggendo AGENTS.md, README.md, docs/SPEC.md, docs/ARCHITECTURE.md, docs/ROADMAP.md, docs/DESIGN_SYSTEM.md e il PDF v0.1, confrontando le sezioni di consegna con modelli, configurazione e servizi correnti. I risultati di prova manuale e i conteggi dei test sono quelli confermati per questa revisione, senza attribuire nuove esecuzioni.

Le sezioni iniziali di alcuni documenti conservano intenzioni originarie: per lo stato corrente questa guida distingue le funzionalità consegnate dai requisiti ancora non realizzati. In particolare, documenta l'effettiva configurazione Vite dei sottodomini Quick Tunnel e i limiti dei flussi tecnico e autenticazione.

Il sorgente mantenibile è SOLVO_Guida_Tecnica_MVP_v0.2.md. L'immagine usa un riferimento relativo al PNG originale nella stessa directory. Le annotazioni `<!-- pagebreak -->` suggeriscono interruzioni per l'esportazione; non alterano il contenuto Markdown. Per rigenerare, renderizzare il Markdown con tabelle, codice e immagini, poi esportare in PDF A4 con margini leggibili, intestazioni conservate e numeri di pagina. Modificare sempre prima il sorgente, evitando editing binario del PDF.

Il file SOLVO_Guida_Tecnica_MVP_v0.1 (2).pdf è conservato senza sovrascrittura. La versione del documento non modifica la versione applicativa, i modelli o le migrazioni.